

Veri Nedir

- Doğru bilgi kümesidir
- Daha küçük parçalara ayrılmayan tek başına bir anlam ifade etmeyip bir başlık altında anlam ifade eden bilgi parçacıkları
- Veri kaydedilebilir grindedir

Veri Tabanı Nedir

- Verinin belirli bir düzende elektronik ortamda saklandığı dosyalardır
- Sistematik erişim imkanı olan, yenilebilir, güncellenebilir, taşınabilir, birbiri arasında ilişki bulunan bilgi kümesidir
 - veriler sistematiktir
 - veriler paylaşılabılır
 - En az bir tablodan oluşmak zorundadır
 - Veritabanı programı oluşturan tablolar veri alanlarında oluşur (data field)

İyi bir veritabanı özellikleri

- Veriler hızlı ve kolay erişilebilir
- Güvenli bir şekilde saklanmalı
- İstenildiği şekilde ve kolay sorgulanmalı

Veritabanı faydaları

- Veri tekrarları ortadan kaldırır veya en aza indirir. Aynı veri tekrarlanmaz
- Bellek alanı israfı önlenir. Aynı veriler farklı tablolarda birden fazla bulunmadığı için gereksiz yer işgal etmez
- Standart sorgu dili kullanmak mümkündür
- Veri bütünlüğünün korunmasını sağlar
- Çoklu güncelleme yapılabilir
- Bilgiler tek bir yerden kontrol edilebilir
- İzersiz bilgiler önlenir

4. İlişkisel veritabanı

- Veriler tablo şeklinde saklanır (operand)
- Bu veritabanında veri alışı-verisi için özel işlem var
- Tablolar Arasında ilişkiler belirtilir
- Bir ilişki bir tabloya bağlıdır tabloda kayıtlı bağlanmaları sağlar
- Günümüzde hemen hemen tüm veri tabanı yönetim sistemleri ilişkisel veritabanına göre yapılır

5. Nesne Veritabanı

- Nesne yönelimli programlamaya dayanan veri modelidir.
- 3 boyutlu bir yapıdan oluşur
- Veriler tek parça halinde gelir

6. Nesne - İlişkisel veritabanı modeli

- Veriler Nesne olarak modellenir ve yoruldur
- Sınıf ve Miras kavramına sahiptir
- Yüksek performanslı
- Küresel bir yapıya sahiptir
- Birden fazla veride tutarlar

Veritabanı Yönetim Sistemi (UTYS)

- yeni veritabanı oluşturmak
- saklamak
- düzenlemek
- geliştirmek
- bakım yapmak

gibi işlemleri yapmaya yarayan yazılım sistem

Kullanıcı ile veritabanı arasında arabirim oluşturur

Veri tabanının tanımlanması = veri tabanını oluşturan verilerin tip ve uzunluğunun belirlenmesidir.

Veri tabanının oluşturulması = Veri için ~~ya~~ belirlenmesi ve saklama ortamına verilerin yüklenmesi

Veri tabanı üzerinde işlem yapmak = belirli bir veri üzerinde sorgu, güncelleme, rapor üretilmesi

Veri tabanının bakım ve sürekliliği = veri tabanına yeni kayıt eklemek, eskiyi çağırma, düzeltme, düzenleme ve silme işlemleri yapmaktır

UYTS **görevleri** =

- Aynı veri tekrar tekrar tutulmaz
- Veri tutarlılığı
- Veri paylılığı / Eş zamanlılık (aynı anda birden fazla erişim)
- Veri bütünlüğü (aynı veri birden fazla yerde düzenleme)
- Veri güvenliği
- Veri Bağımsızlığı

Veri tabanı kullanıcıları

→ **Son kullanıcılar**

- Sıradan kullanıcılar
- Veri tabanına formlar veya uygulama ile erişilir
- DML kullanarak sorgu üretiliyorlar

→ **Uygulama Programcısı**

- Veri tabanına erişimlik uygulama tarafların

→ **Veri tabanı yöneticisi**

- Kurumsal senaryo taraflar ve yönetir
- Uygulama ve kullanıcı performansını belirler
- UYTS bakımını, izler ve düzenler
- Veri tabanının yedek ve yenisinden bakılır
- Güvenlik ve güvenilirlikten sorumlu

UYTS Bileşenleri=

- Erişim ve veri işleme
- Genel amaçlı bir sorgu dili (SQL)
- Genel amaçlı güvenlik sistemi
- Yedekleme ve diğer yardımcı birimler
- Uygulama geliştirme ortamı
- Rapor üretici

Veritabanı Sistemleri=

- Microsoft Access
- MySQL
- IBM DB2
- Interbase
- Informix
- PostgreSQL
- Microsoft SQL server
- PostgreSQL
- Oracle

slot 44

Hangi Veritabanı

küçük yoğunlukta trafik = MySQL

Web uygulamaları için = MySQL

Daha büyük ve orta ölçekli = PostgreSQL, PostgreSQL
MS SQL

Oracle ise çok yüksek güvenilirlik ve hız gücü

Temel Kavramlar

Veritabanı = içinde tabloların bulunduğu sistem

Tablo = Veritabanında bilgilerin saklandığı alt birim

Alan (field) = Tablonun sütunlarıdır. Bu sayede veri tipi sınırlanır

Kayıt (Record) = Tablonun her bir satırıdır

Veri tipi = Kayıtların yapılarının tanımlanması için kullanılır

İlişkilendirme = Tek sorgu ile birden fazla bilgi

alma işlemi

Tablolar =

- Veritabanının temelini oluşturur
- Satır ve sütunlardan oluşur
- Aynı konu ile ilgili olan bilgiler birleştirilir
- Aynılar Aynı tablo
- Veri tekrarı olmamalı
- Gereksiz alan kullanılmamalı
- Alanlar basit indirgenmeli

Anahtar

Tabloda her kayıta diğer kayıtlardan farklı aynı sütunda aynı değere sahip olmayan bir alanın bulunması gerekir. Bu alan için anahtar tanımlanması gerekir

- Primary key
- unique key
- foreign key

Primary Key

- Tanımlandığı Alanda her kayıta farklı olduğunu belirler
- Bir tabloda verilerin hangi alana göre sıralanacağını belirler
- Birden fazla alan birbirine primary key oluşturur
- Tanımlı alana veri girişi zorunlu

→ Otomatik olarak birincil anahtar

→ Tek alan birincil anahtar

→ Birden fazla alan birincil anahtar

İlişki = Veritabanı arasında kurulan fiziksel veya mantıksal bağlantılar temsil eden yapılara denir

Veritabanı Arası İlişki 87 - 116

- Bir den bir e
- Bir den çoğa
- Çok den bir e
- Çok den çoğa

Veritabanı Normalizasyonu

- İlişkisel veri tabanı tasarımı sırasında
- verinin tekrarlardan
 - kaybını veya yetersizliğini önlemek
 - Veri bütünlüğünü sağlamak için kullanılır

Amaçları =

1. Veri bütünlüğünü sağlamak
2. Uygulamadan Bağımsızlık
3. Performans artırmak

1. Veri Bütünlüğü

Gerekli veri tekrarlama, engellenir

2. Uygulamadan Bağımsızlık

- Veritabanı onu kullanan uygulama değişse bile sorunsuz çalışır
- Uygulama mantıksal model üzerinde değil, fiziksel model üzerine etki yapar

The SQL INSERT INTO Statement

- **CREATE DATABASE** : Creates a new database.
- **ALTER DATABASE** : Changes the property of a database.
- **CREATE TABLE** : Creates a new table.
- **ALTER TABLE** : Changes the property of a table.
- **DROP TABLE** : Completely deletes a table.
- **CREATE INDEX** : Creates an index.
- **DROP INDEX** : Deletes the index

• INSERT INTO Syntax,

- **INSERT INTO** *table_name* (*column1*, *column2*, *column3*, ...)
VALUES (*value1*, *value2*, *value3*, ...);
- **INSERT INTO** *table_name*
VALUES (*value1*, *value2*, *value3*, ...);

SELECT Statement

Syntax

- **SELECT** *column1*, *column2*, ...
FROM *table_name*;
- **SELECT** * **FROM** *table_name*;

• UPDATE Syntax

- **UPDATE** *table_name*
SET *column1* = *value1*, *column2* = *value2*, ...
WHERE *condition*;

- ORDER BY Syntax,
- `SELECT column1, column2, ...
FROM table_name
ORDER BY column1, column2, ... ASC|DESC;`
- ORDER BY Example,
- `SELECT * FROM Customers
ORDER BY CustomerName DESC;`

MIN() Syntax & MAX() Syntax

- `SELECT MIN(column_name)
FROM table_name
WHERE condition;`
- `SELECT MAX(column_name)
FROM table_name
WHERE condition;`

The SQL COUNT(), AVG() and SUM() Functions

- `SELECT COUNT(column_name)
FROM table_name
WHERE condition;`
- `SELECT AVG(column_name)
FROM table_name
WHERE condition;`
- `SELECT SUM(column_name)
FROM table_name
WHERE condition;`



SQL LIKE Examples

- The following SQL statement selects all customers with a CustomerName starting with "a":
- `SELECT * FROM Customers
WHERE CustomerName LIKE 'a%';`
- The following SQL statement selects all customers with a CustomerName ending with "a":
- `SELECT * FROM Customers
WHERE CustomerName LIKE '%a';`



AND & OR & NOT Syntax

- `SELECT column1, column2, ...`
`FROM table_name`
`WHERE condition1 AND condition2 AND condition3 ...;`
- `SELECT column1, column2, ...`
`FROM table_name`
`WHERE condition1 OR condition2 OR condition3 ...;`
- `SELECT column1, column2, ...`
`FROM table_name`
`WHERE NOT condition;`

Alias Column & Table Syntax

Gecici isim vermeyi sağlar

Alias Column Syntax

- `SELECT column_name AS alias_name`
`FROM table_name;`
- `SELECT name_of_table_or_column AS name_of_alias`

Alias Table Syntax

- `SELECT column_name(s)`
`FROM table_name AS alias_name;`

Create Table

Example,

- `CREATE TABLE Persons (`
`PersonID int,`
`LastName varchar(255),`
`FirstName varchar(255),`
`Address varchar(255),`
`City varchar(255)`
`);`

PersonID	LastName	FirstName	Address	City

Example,

- `CREATE TABLE cars(make varchar(10))`
`INSERT INTO cars VALUES ('HONDA');`
`INSERT INTO cars VALUES ('BMW');`
`INSERT INTO cars VALUES ('BMW');`
`INSERT INTO cars VALUES ('BMW');`
`INSERT INTO cars VALUES ('AUDI');`
`INSERT INTO cars VALUES ('AUDI');`